SPRING BOOT HANDBOOK

BOOK - PART 1

1. WHAT IS SPRING & SPRING BOOT ?

⇒ What is Spring ?

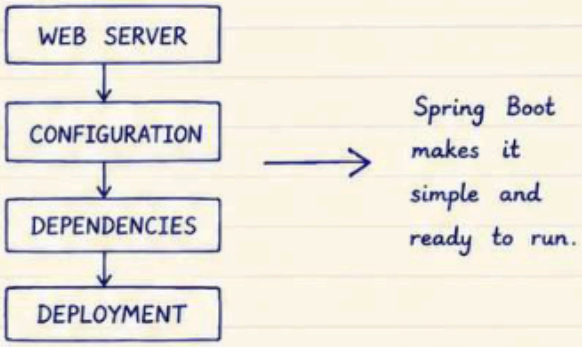
- Spring is a lightweight Java framework.
- It provides a comprehensive programming and configuration model for modern Java based enterprise applications.
- It is built on core concepts like IoC, DI and AOP.

Why Spring Boot ?

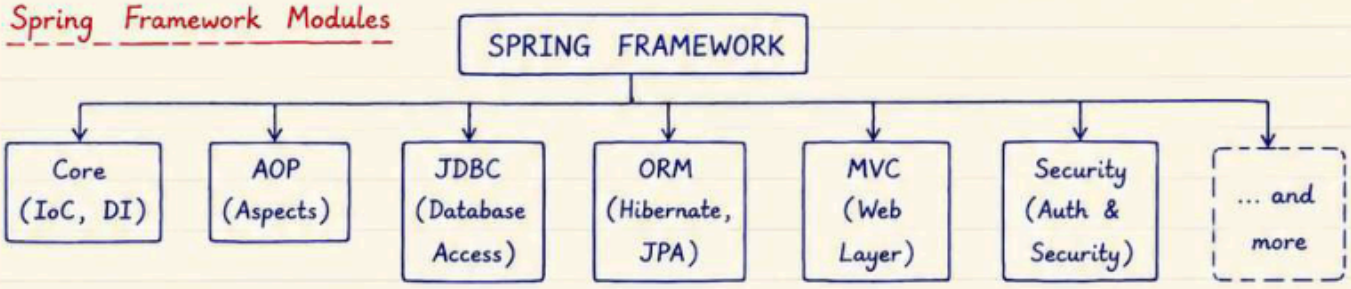
- Faster development
- Auto configuration
- Embedded servers
- Production ready
- Less XML, more Annotations
- Easy Dependency Management
- Built-in Actuator & Monitoring

⇒ What is Spring Boot ?

- Spring Boot is an extension of Spring framework.
- It simplifies development by auto-configuration and convention over configuration.
- It helps to create stand-alone, production ready applications with minimal setup.
- It has embedded servers (Tomcat, Jetty, Undertow) and reduces boilerplate code.



Spring Framework Modules



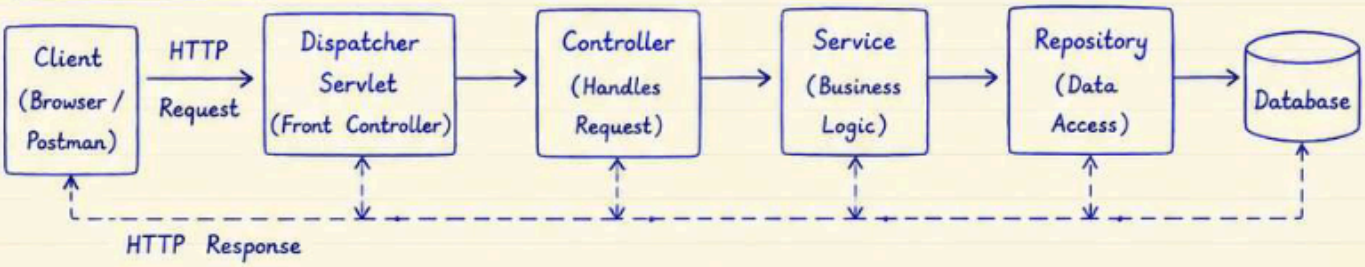
Spring vs Spring Boot

Feature	Spring Framework	Spring Boot
Configuration	Manual configuration	Auto configuration
Setup	Complex	Simple
Server	External (Tomcat, Jetty etc.)	Embedded (Tomcat, Jetty etc.)
Deployment	WAR file	JAR file
Development	Time consuming	Very fast
XML Usage	More XML	Less / No XML
Monitoring	Manual setup	Built-in Actuator
Main Goal	Provide full control	Convention over configuration

2. SPRING BOOT ARCHITECTURE

Spring Boot follows a layered architecture. Each layer has a specific responsibility and they work together to handle a request.

Architecture Flow :



Components Description :

- Client : Sends HTTP request and receives response.
- DispatcherServlet : Front controller of Spring MVC. It receives the request and dispatches it to the appropriate controller.
- Controller : Handles the request, validates input and returns response.
- Service : Contains business logic and calls repository.
- Repository : Interacts with database to perform CRUD operations.
- Database : Stores application data.

Example Flow :

Client → DispatcherServlet → Controller → Service → Repository → Database

Database → Repository → Service → Controller → DispatcherServlet → Client

Example Code (Controller) :

```
@RestController                                  → Marks this class as REST Controller
@RequestMapping("/api/users")                    → Base URL for all APIs in this controller
public class UserController {
    @Autowired
    private UserService userService;             → Injects UserService dependency

    @GetMapping("/{id}")                         → Maps GET request with user id
    public User getUser(@PathVariable Long id) {
        return userService.getUser(id);          → Calls service layer and returns data
    }
}
```

Example (application.properties) :

```
server.port = 8080
spring.datasource.url = jdbc:mysql://localhost:3306/demo
spring.datasource.username = root
spring.datasource.password = root
spring.jpa.hibernate.ddl-auto = update
spring.jpa.show-sql = true
```

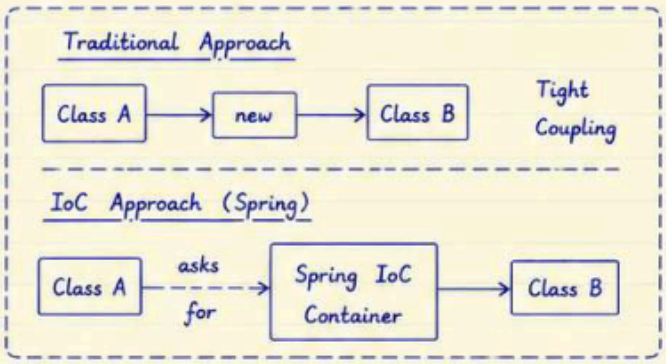
} Basic configuration for database & server

This flow is the backbone of any Spring Boot web application.

3. IOC CONTAINER & DEPENDENCY INJECTION

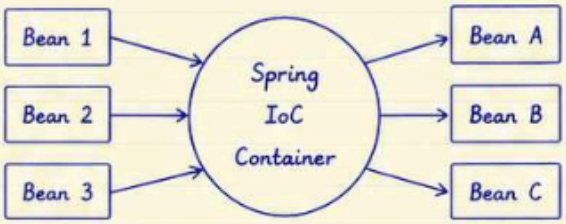
3.1 What is IoC ?

- IoC (Inversion of Control) is a principle where the control of object creation and management is transferred from application code to the Spring container.
- The container creates objects, wires them together, manages their lifecycle and dependencies.



3.2 Spring IoC Container

- Spring IoC container is responsible for creating, configuring and assembling the beans.
- The container provides the beans to the application when required.



3.3 Dependency Injection (DI)

- DI is a design pattern where the container injects the required dependencies into the objects.
- It helps in achieving loose coupling and easy unit testing.

Types of DI

- Constructor Injection
- Setter Injection
- Field Injection

Example (Constructor Injection)

```
public class UserService {
    private UserRepository userRepository;

    public UserService(UserRepository repo) {
        this.userRepository = repo;
    }
}
```

3.4 Example Code

1. Bean Class

```
@Repository
public class UserRepository {
    public String getUserById(int id) {
        return "User" + id;
    }
}

@Service
public class UserService {
    private UserRepository userRepository;

    // Constructor Injection
    public UserService(UserRepository repo) {
        this.userRepository = repo;
    }
    public String getUser(int id) {
        return userRepository.getUserById(id);
    }
}
```

2. Configuration (Java Based)

```
@Configuration
public class AppConfig {

    @Bean
    public UserRepository userRepository() {
        return new UserRepository();
    }

    @Bean
    public UserService userService(
            UserRepository userRepository) {
        return new UserService(userRepository);
    }
}
```

3. Main Class

```
public class MainApplication {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext
            context = new
            AnnotationConfigApplicationContext
                (AppConfig.class);

        UserService service =
            context.getBean(UserService.class);

        System.out.println(service.getUser(1));

        context.close();
    }
}
```

3.5 Key Points

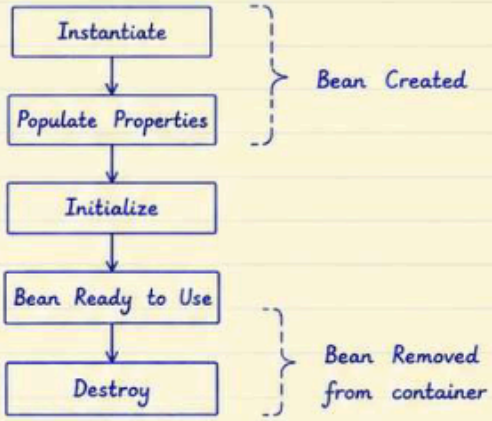
- IoC reduces tight coupling between classes.
- DI makes code more testable and maintainable.
- Spring container manages the lifecycle of beans.
- We just declare our dependencies, container takes care of creation and injection.

4. BEAN LIFECYCLE & BEAN SCOPE

4.1 Bean Lifecycle

Spring manages the complete lifecycle of a bean.
The steps are :

① Instantiate → Container creates bean instance.
② Populate → Set values to the bean properties.
③ Initialize → Call BeanNameAware, InitializingBean, @PostConstruct etc.
④ Ready → Bean is ready to use.
⑤ Destroy → Call @PreDestroy, DisposableBean before removing bean.



4.2 Example of Lifecycle

```
public class DemoBean implements InitializingBean, DisposableBean {
    public DemoBean() {
        System.out.println("1. Constructor");
    }

    @Override
    public void afterPropertiesSet() {
        System.out.println("2. afterPropertiesSet()");
    }
    @PreDestroy
    public void destroy() {
        System.out.println("5. @PreDestroy");
    }
}
```

Output Order :

1. Constructor
2. afterPropertiesSet()
3. Bean is ready to use
4. Bean will do its work
5. @PreDestroy

Note : Steps 1 and 2 are done by Spring container for every bean.

4.3 Bean Scopes

Scope	Description	Where it is used	Annotation / Config
Singleton	Only one instance per Spring IoC container.	Default scope.	@Scope("singleton") (default)
Prototype	New instance for every request.	When each request needs separate object.	@Scope("prototype")
Request	One instance per HTTP request.	Web applications.	@Scope("request")
Session	One instance per HTTP session.	Web applications.	@Scope("session")
Application	One instance per ServletContext.	Web applications.	@Scope("application")
WebSocket	One instance per WebSocket session.	WebSocket applications.	@Scope("websocket")

Example :

```
@Component
@Scope("prototype")
public class Student {
    // class body
}
```

→ Every time container is asked for Student bean, it will create new object.

5. SPRING ANNOTATIONS

5.1 Common Stereotype Annotations

Annotation	Used For	Description	Example
@Component	General component	Marks a class as a Spring managed component.	@Component public class EmailUtil { }
@Service	Service layer	Indicates that the class belongs to service layer.	@Service public class UserService { }
@Repository	Data access layer	Indicates that the class is responsible for database access.	@Repository public interface UserRepository extends JpaRepository<User,Long>{}
@Controller	Web MVC controller	Marks a class as Spring MVC Controller. Returns view name.	@Controller public class HomeController { }
@RestController	REST API controller	Combination of @Controller and @ResponseBody.	@RestController public class UserController { }
@Configuration	Java based configuration	Indicates that the class provides bean definitions.	@Configuration public class AppConfig { }

@Bean	Bean definition method	Marks a method as a bean creator in @Configuration class.	@Bean
			public DataSource dataSource(){
			return new HikariDataSource();
			}

5.2 AutoWiring

- AutoWiring is used to inject the bean dependencies automatically.
- Types : byType (default), byName, constructor, no

```
@Service
public class UserService {
    // dependency
    @Autowired          // byType (default)
    private UserRepository userRepository;
}
```

Other ways :

- byName → matches by property name
- constructor → injects through constructor
- no → no autowiring

5.3 Component Scan

- Spring Boot automatically scans the packages under the main class package and its sub-packages.
- It detects @Component, @Service, @Repository, @Controller etc. and registers them as beans.

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

→ This annotation includes @ComponentScan, @Configuration and @EnableAutoConfiguration.

5.4 Other Important Annotations

- @Value → Injects values from application.properties / application.yml
- @Qualifier → Used with @Autowired to resolve ambiguity.
- @Primary → Marks one bean as primary when multiple beans of same type exist.
- @Lazy → Initializes the bean only when it is required.

6. REST API BASICS IN SPRING BOOT

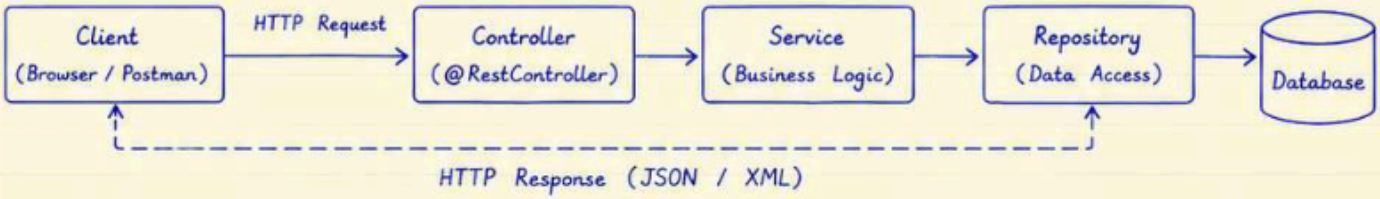
6.1 What is REST API ?

- REST (Representational State Transfer) is an architectural style for designing networked applications.
- It uses HTTP methods to perform CRUD operations on resources.
- Spring Boot makes it very easy to build RESTful web services.

CRUD using HTTP Methods

HTTP Method	Operation	Description
GET	Read	Fetch data
POST	Create	Create new resource
PUT	Update	Update entire resource
PATCH	Partial Update	Update partial data
DELETE	Delete	Delete resource

6.2 REST API Flow



6.3 Example – Simple REST Controller

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return new User(id, "Aman", "aman@example.com");
    }
    @PostMapping
    public User createUser(@RequestBody User user) {
        user.setId(101L);
        return user;
    }
    @PutMapping("/{id}")
    public User updateUser(@PathVariable Long id,
                           @RequestBody User user) {
        user.setId(10L);
        return user;
    }
    @DeleteMapping("/{id}")
    public String deleteUser(@PathVariable Long id) {
        return "User deleted with id: " + id;
    }
}
```

Explanation :

- @RestController = @Controller + @ResponseBody
- @RequestMapping defines base URL for the class
- @GetMapping handles GET request
- @PostMapping handles POST request
- @PutMapping handles PUT request
- @DeleteMapping handles DELETE request
- @PathVariable fetches value from URL
- @RequestBody binds request body to object

Sample JSON (Request Body)

```
{
    "name": "Aman",
    "email": "aman@example.com",
    "age": 25
}
```

6.4 Response Example

GET /api/users/1 ⟶ Response (200 OK)

```
{
    "id": 1,
    "name": "Aman",
    "email": "aman@example.com"
}
```

POST /api/users ⟶ Response (201 Created)

```
{
    "id": 101,
    "name": "Aman",
    "email": "aman@example.com"
}
```

7. SPRING BOOT PROJECT STRUCTURE

A typical Spring Boot project follows a standard folder structure.

```
src
├── main
│   └── java
│       └── com.example.demo
│           ├── controller   → REST Controllers
│           ├── service      → Service classes (business logic)
│           ├── repository   → JPA Repositories
│           ├── entity       → Entity classes (Database tables)
│           ├── dto          → Request / Response DTOs
│           ├── config       → Configuration classes
│           └── exception    → Custom exceptions & handlers
│   └── resources
│       ├── application.properties → App configuration
│       ├── static           → Static files (css, js, images)
│       └── templates        → Thymeleaf templates (if using)
└── test
    └── java
        └── com.example.demo
            └── DemoApplicationTests → Unit & Integration tests
```

Important Files

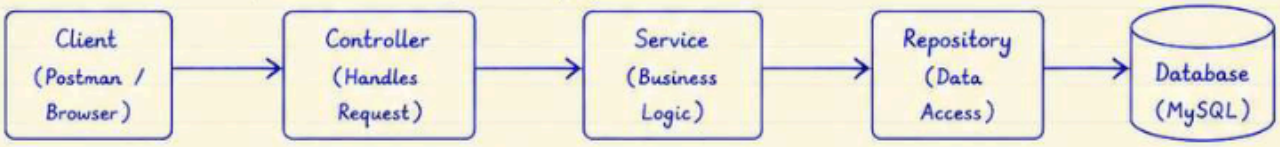
- DemoApplication.java → Main class
- application.properties → Configuration
- pom.xml → Dependencies
- build.gradle → Build file

Main Class Example

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

8. BUILD FIRST CRUD API – OVERVIEW FLOW

We will build a simple CRUD API for managing Student data.



CRUD Endpoints

- Create - POST /api/students
- Read All - GET /api/students
- Read By Id - GET /api/students/{id}
- Update - PUT /api/students/{id}
- Delete - DELETE /api/students/{id}

Base URL

http://localhost:8080/api/students

Technology Used

- Spring Boot
- Spring Web
- Spring Data JPA
- MySQL
- Lombok (optional)
- Validation
- Postman (for testing)

Dependencies in pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope>
</dependency>
```

9. APPLICATION PROPERTIES & CONFIGURATION

9.1 What is application.properties ?

- It is the main configuration file in Spring Boot. We use it to configure database, server port, logging, and other properties.

How it works



9.2 Where is it located ?

- src/main/resources/application.properties

9.3 Example Properties

```
server.port = 8080

spring.application.name = demo-app

spring.datasource.url = jdbc:mysql://localhost:3306/demo
spring.datasource.username = root
spring.datasource.password = root123
spring.jpa.hibernate.ddl-auto = update
spring.jpa.show-sql = true

logging.level.org.springframework = INFO
logging.level.com.example.demo = DEBUG
```

9.4 Accessing Properties in Code

Using @Value

```
@Value("${app.name}")
private String appName;

@Value("${server.port}")
private int port;
```

Using @ConfigurationProperties

```
@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {
    private String name;
    private String description;
    // getters and setters
}
```

9.5 Custom Properties Example

application.properties

```
app.name = Student Service
app.description = This is a demo application
```

→

```
@Component
@ConfigurationProperties(prefix = "app")
public class AppProperties {
    private String name;
    private String description;
    // getters and setters
}
```

9.6 Profile based Properties

We can have different properties for different environments.

```
└── resources
    ├── application.properties        → (default)
    ├── application-dev.properties    → (for dev environment)
    └── application-prod.properties   → (for prod environment)
```

Activate Profile

In application.properties

spring.profiles.active = dev

(or)

Run from command line

--spring.profiles.active = dev

10. SPRING BOOT RUN & TEST

10.1 Run the Application

- We can run the Spring Boot application in multiple ways.

Method	Command
1. From IDE	Right click on DemoApplication.java → Run 'DemoApplication'
2. Using Maven	mvn spring-boot:run
3. Using Gradle	gradle bootRun
4. Using JAR	mvn clean package java -jar target/demo-app.jar

Run Application
(e.g., http://localhost:8080)
↓
Spring Boot
Starts Embedded Server
(Tomcat)
↓
Application Loaded
& Ready
↓
API Endpoints
Available

10.2 Sample API Endpoints (from our project)

Method	Endpoint	Description	Sample Response
GET	/api/students	Get all students	[{"id":1,"name":"Aman"}, ...]
GET	/api/students/1	Get student by id	{"id":1,"name":"Aman","age":25}
POST	/api/students	Create new student	{"id":101,"name":"Aman","age":25}
PUT	/api/students/1	Update student	{"id":1,"name":"Aman","age":26}
DELETE	/api/students/1	Delete student	{"message":"User deleted"}

10.3 Testing the APIs

- We can test APIs using Postman / Browser / cURL

Example : GET /api/students/1

Request (GET)

http://localhost:8080/api/students/1 →

Response (200 OK)

```
{
  "id": 1,
  "name": "Aman",
  "email": "aman@example.com",
  "age": 25
}
```

Example cURL (GET)

```
curl -X GET http://localhost:8080/api/students/1
  -H "Accept: application/json"
```

Example cURL (POST)

```
curl -X POST http://localhost:8080/api/students
  -H "Content-Type: application/json"
  -d "{\"name\":\"Aman\",\"email\":\"aman@example.com\",\"age\":25}"
```

10.4 Common Application Properties

Property	Use
server.port = 8080	Change application port
spring.application.name = demo-app	Set application name
spring.datasource.url = jdbc:mysql://localhost:3306/demo	Database connection URL
spring.datasource.username = root	Database username
spring.datasource.password = 123	Database password
spring.jpa.hibernate.ddl-auto = update	Auto create / update tables
logging.level.org.springframework = INFO	Set logging level

What we covered in Part - 1

1. Spring Boot Overview
2. Project Structure
3. Maven Dependencies
4. Bean Lifecycle & Scope
5. Spring Annotations
6. REST API Basics
7. Project Structure & CRUD Flow
8. Application Properties
9. Run & Test Application

Next → Advanced Topics in Part - 2